

Structure d'un skill

Un skill est un dossier dans `.claude/skills/` contenant un fichier `SKILL.md` et optionnellement des ressources supplémentaires (docs, templates, scripts, checklists).

```
mon-skill/
├─ SKILL.md           # Instructions + frontmatter (obligatoire)
├─ reference.md       # Documentation de référence
├─ checklists/
│   └─ quality.md
│   └─ examples/
└─ pattern.ts
```

Modes d'invocation

Mode	Frontmatter	Comportement
User-invocable	<code>disable-model-invocation: true</code>	Déclenché par l'utilisateur via <code>/nom</code>
Model-invocable	(aucun flag)	Chargé automatiquement par le modèle
Les deux	(aucun flag)	Invocable des deux façons

Frontmatter SKILL.md

```
---
name : security-guardian
description : Expertise OWASP, auth, et protection des données
allowed-tools : Read Grep Bash
---
```

Champ	Rôle
<code>name</code>	Kebab-case, 1-64 chars, doit correspondre au dossier
<code>description</code>	Déclencheur d'activation (max 1024 chars)
<code>allowed-tools</code>	Whitelist d'outils (supporte les wildcards)
<code>argument-hint</code>	Placeholder dans le menu si le skill accepte <code>\$ARGUMENTS</code>
<code>disable-model-invocation</code>	<code>true</code> pour invocation manuelle uniquement

Skills vs Commands

Aspect	Skill	Command
Ressources	Peut embarquer des fichiers	Markdown seul
Portée	Connaissance domaine	Workflow procédural
Réutilisation	Entre agents	Invocation directe
Standard	agentskills.io (portable)	Claude Code uniquement

La différence principale : un skill peut embarquer un fichier `reference.md` de 500 lignes qui devient disponible lors de l'invocation. Une command ne peut pas.

Marketplace : npx add-skill

```
# Installer le bundle Vercel (3 skills)
npx add-skill vercel-labs/agent-skills

# Installer le skill Supabase
npx add-skill supabase/agent-skills

# Vérifier les skills installés
ls ~/.claude/skills/
```

Les skills du marketplace passent un audit de sécurité automatique en 3 couches (Socket, Snyk, Gen) avant publication. Plus de 200 skills disponibles.

Invocation

Les skills se déclenchent comme les commandes, par leur nom :

```
/security-guardian # Invocation directe
/tdd # Skill TDD
/design-patterns detect # Avec argument
```

Les agents peuvent précharger des skills via le champ `skills:` dans leur frontmatter — le contenu est injecté dans le contexte de l'agent au démarrage.

Lifecycle et amélioration

Deux patterns de gestion dans le temps :
Claudeception — méta-skill qui génère de nouveaux skills à partir des découvertes de session. Install :
`git clone https://github.com/blader/Claudeception ~/.claude/skills/claudeception`

Claude Reflect System — améliore les skills existants en analysant les corrections détectées pendant les sessions. Propose les changements avant de les appliquer.

Précaution importante

Les skills ne sont invoqués que lors d'une activation explicite — les evals montrent une invocation automatique dans seulement 56% des cas. Pour des instructions critiques qui doivent toujours s'appliquer, utiliser `CLAUDE.md` ou `.claude/rules/` plutôt qu'un skill.